

Características de Qualidade de Repositórios Java no GitHub

Lab02 — Experimentação de Software

Autor

Kaio Henrique Oliveira da Silveira Barbosa

Índice

Índice	2
1. Introdução	3
2. Questões de Pesquisa e Hipóteses	3
3. Metodologia	4
3.1 Materiais e Ferramentas	4
3.2 Definição de Métricas	4
3.3 Fluxo de Execução	5
3.4 Desafios e Tomadas de Decisão	6
4. Resultados	7
4.1 RQ01 — Popularidade vs. Qualidade	7
4.2 RQ02 — Maturidade vs. Qualidade	8
4.3 RQ03 — Atividade vs. Qualidade	8
4.4 RQ04 — Tamanho vs. Qualidade	9
5. Discussão e Visualização dos Resultados	11
5.1 RQ01 — Popularidade vs. Qualidade	11
5.2 RQ02 — Maturidade vs. Qualidade	12
5.3 RQ03 — Atividade vs. Qualidade	13
5.4 RQ04 — Tamanho vs. Qualidade	14
6. Conclusão	17

1. Introdução

O desenvolvimento colaborativo de software open-source apresenta desafios inerentes à manutenção da qualidade interna do código. À medida que um projeto cresce em popularidade e recebe contribuições de múltiplos desenvolvedores, atributos como modularidade, coesão e manutenibilidade tornam-se difíceis de preservar. Práticas como revisão de código e análise estática via ferramentas de CI/CD buscam mitigar esses riscos.

Este trabalho analisa empiricamente a qualidade interna de repositórios Java hospedados no GitHub, correlacionando métricas de produto — calculadas pela ferramenta CK — com características do processo de desenvolvimento de cada projeto. O estudo parte da hipótese geral de que projetos mais populares, maduros e ativos tendem a apresentar melhores indicadores de qualidade interna, dado o maior escrutínio da comunidade sobre o código.

A análise foi conduzida sobre uma amostra dos top-1.000 repositórios Java por número de estrelas, coletados via API GraphQL do GitHub. Após o processamento, 978 repositórios foram analisados com sucesso — os demais foram excluídos por limitações técnicas descritas na Seção 3.4.

2. Questões de Pesquisa e Hipóteses

Para orientar a investigação, foram formuladas quatro questões de pesquisa, cada uma relacionando uma característica do processo de desenvolvimento com as métricas de qualidade obtidas pelo CK. A tabela abaixo apresenta as questões, as variáveis de processo correspondentes e as hipóteses informais estabelecidas antes da coleta de dados.

RQ	Variável de Processo	Hipótese Informal	Resultado
RQ01	Popularidade (estrelas)	Projetos mais populares terão menor CBO e maior coesão (LCOM), dado maior volume de revisões externas.	Não confirmada
RQ02	Maturidade (idade em anos)	Projetos mais antigos possuem arquitetura mais consolidada, mas podem acumular dívida técnica — DIT ligeiramente maior e LCOM variável.	Parcialmente confirmada
RQ03	Atividade (nº de releases)	Projetos com ciclos de release frequentes indicam manutenção ativa, favorecendo melhores CBO e LCOM.	Não confirmada
RQ04	Tamanho (LOC + comentários)	Projetos maiores tendem a ser mais complexos, com CBO e DIT mais altos; muitos comentários podem indicar código legado.	Confirmada

Tabela 1 — Questões de pesquisa, variáveis de processo e hipóteses informais.

3. Metodologia

3.1 Materiais e Ferramentas

O experimento foi conduzido inteiramente por meio de scripts Python, utilizando as ferramentas listadas na tabela a seguir. A escolha do Python se justifica pela disponibilidade de bibliotecas adequadas para coleta de dados via API, análise estatística e geração de visualizações. A ferramenta CK (versão 0.7.0) foi responsável pela análise estática do código-fonte Java, gerando arquivos .csv com as métricas por classe.

Ferramenta / Recurso	Finalidade
Python 3.x	Scripts de coleta, processamento e análise
GitHub GraphQL API	Coleta dos 1.000 repositórios Java
CK 0.7.0	Análise estática de métricas de qualidade Java
Git (--depth=1)	Clonagem superficial dos repositórios
matplotlib	Geração de gráficos de dispersão
scipy	Cálculo da correlação de Spearman
python-dotenv	Gerenciamento do token de acesso ao GitHub

Tabela 2 — Ferramentas e recursos utilizados no experimento.

3.2 Definição de Métricas

O estudo emprega dois conjuntos de métricas: métricas de processo, que caracterizam o repositório em termos de desenvolvimento, e métricas de qualidade, calculadas pela análise estática do código-fonte. As tabelas abaixo detalham cada conjunto.

Métricas de Processo

Métrica	Campo	Descrição
Popularidade	stars	Número de estrelas no GitHub
Maturidade	age_years	Idade em anos calculada a partir de created_at
Atividade	releases	Número total de releases publicadas
Tamanho — código	loc_median	Mediana de linhas de código por classe (via CK)
Tamanho — comentários	comment_lines	Total de linhas de comentário Java (// e /* */)

Tabela 3 — Métricas de processo coletadas via API do GitHub.

Métricas de Qualidade (via CK)

Sigla	Nome Completo	Interpretação
CBO	Coupling Between Objects	Acoplamento entre classes — valores altos indicam maior dependência entre módulos

DIT	Depth Inheritance Tree	Profundidade da herança — valores altos indicam hierarquias complexas
LCOM	Lack of Cohesion of Methods	Falta de coesão — valores altos indicam classes com responsabilidades difusas

Tabela 4 — Métricas de qualidade calculadas pela ferramenta CK.

Cada métrica de qualidade é sumarizada por repositório com mediana, média e desvio padrão. A mediana é adotada como medida principal por ser robusta à presença de outliers, característica importante em uma amostra com alta variabilidade entre os projetos analisados.

3.3 Fluxo de Execução

O processo de coleta e análise foi estruturado em quatro etapas sequenciais, conforme ilustrado na Figura 1. A etapa de coleta utiliza a API GraphQL do GitHub para obter os repositórios mais populares em Java. Em seguida, cada repositório é clonado superficialmente para execução do CK. Os resultados individuais são consolidados em um único arquivo e, por fim, a análise estatística gera os gráficos de correlação e os valores de Spearman.

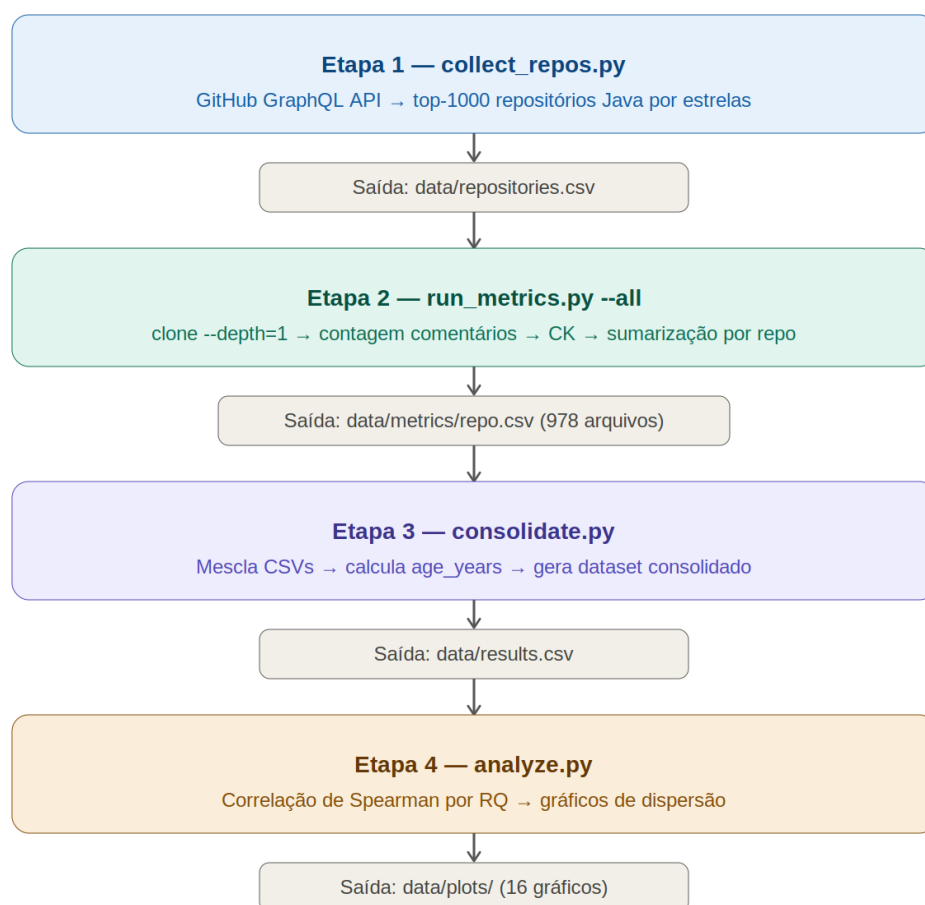


Figura 1 — Fluxo de execução do experimento: coleta → métricas → consolidação → análise.

Etapa 1 — collect_repos.py

Consulta a API GraphQL do GitHub em páginas de 50 repositórios (first: 50) e coleta nome, URL, estrelas, data de criação e total de releases. Saída: data/repositories.csv.

Etapa 2 — run_metrics.py --all

Para cada repositório: (a) executa `git clone --depth=1`; (b) conta linhas de comentário via `regex`; (c) executa o CK para gerar `class.csv`; (d) sumariza as métricas por repositório. Saída: data/metrics/<repo>.csv.

Etapa 3 — consolidate.py

Mescla todos os CSVs individuais, calcula `age_years` a partir de `created_at` e gera o conjunto final. Saída: data/results.csv.

Etapa 4 — analyze.py

Calcula a correlação de Spearman entre cada par (variável de processo × métrica de qualidade) e gera os gráficos de dispersão. Saída: data/plots/.

3.4 Desafios e Tomadas de Decisão

Ao longo da execução do experimento, quatro desafios relevantes foram identificados e tratados. Cada um deles implicou uma decisão de projeto que afetou a amostra final ou a implementação dos scripts.

Desafio 1 — Ausência de linhas de comentário no CK

O CK 0.7.0 não expõe linhas de comentário em seu output. **Decisão:** implementar a contagem diretamente no script `run_metrics.py`, percorrendo os arquivos `.java` com expressões regulares para identificar comentários de linha (`//`) e de bloco (`/* ... */`).

Desafio 2 — Incompatibilidade do CK com Java moderno

O CK 0.7.0 utiliza o Eclipse JDT internamente. Repositórios com sintaxe Java 14+ (`switch expressions`, `records`, `sealed classes`) causam `crash` com `NullPointerException` ou `ArrayIndexOutOfBoundsException`. **Decisão:** registrar como limitação e excluir os 22 repositórios afetados. A amostra resultante de 978 repositórios permanece representativa.

Desafio 3 — Falha na remoção de clones no macOS

Repositórios grandes (ex: `apache/tika` com 415 MB) contêm arquivos com atributos estendidos do macOS que impedem a remoção via `shutil.rmtree()`, resultando em `[Errno 66] Directory not empty`. **Decisão:** tratar como erro não-crítico — as métricas foram coletadas com sucesso antes da limpeza, portanto os CSVs gerados são válidos.

Desafio 4 — Erro 502 na API GraphQL do GitHub

A query inicial utilizava `first: 100` por página. O campo `releases { totalCount }` tornava a requisição pesada o suficiente para gerar erro 502 consistentemente. **Decisão:** reduzir o tamanho da página para `first: 50`, eliminando completamente o problema.

4. Resultados

Para cada questão de pesquisa, os resultados são apresentados por meio de (i) estatísticas descritivas da variável de processo, (ii) tabela de correlação de Spearman entre a variável de processo e cada métrica de qualidade, e (iii) discussão e visualização dos dados, integrada na mesma seção. A mediana é adotada como medida central principal.

4.1 RQ01 — Popularidade vs. Qualidade

A popularidade dos repositórios é mensurada pelo número de estrelas no GitHub. Antes de analisar os resultados, a hipótese formulada era de que projetos com maior visibilidade tendem a receber mais revisões externas, o que favoreceria uma arquitetura mais cuidadosa — traduzindo-se em menor CBO e menor LCOM. A tabela abaixo apresenta as estatísticas descritivas do número de estrelas na amostra.

Medida	Valor
Mediana	5.793
Média	9.637,95
Desvio Padrão	11.782,20

Tabela 5 — Estatísticas descritivas: número de estrelas ($n = 978$).

A alta assimetria evidenciada pelo desvio padrão muito superior à mediana indica que poucos repositórios concentram a maior parte das estrelas. Essa heterogeneidade é visível nos gráficos de dispersão apresentados a seguir. A tabela abaixo apresenta os resultados da correlação de Spearman.

Métrica de Qualidade	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,0296	0,3570	Não
DIT (mediana)	-0,0391	0,2232	Não
LCOM (mediana)	0,0076	0,8129	Não

Tabela 6 — Correlação de Spearman: estrelas $\times$ métricas de qualidade.

Nenhuma das três correlações foi estatisticamente significativa ($p > 0,05$ em todos os casos), com valores de ρ próximos de zero. Os gráficos de dispersão confirmam visualmente a ausência de qualquer padrão entre estrelas e CBO, DIT ou LCOM. A hipótese inicial é portanto refutada: popularidade no GitHub mede visibilidade e adoção, não arquitetura interna. Repositórios amplamente estrelados incluem desde frameworks maduros até materiais educacionais (tutoriais, exemplos de código), categorias essencialmente distintas em termos de qualidade estrutural.

Nota: os gráficos de dispersão para todas as RQs encontram-se na Seção 5.

4.2 RQ02 — Maturidade vs. Qualidade

A maturidade de um repositório é representada pela sua idade em anos, calculada a partir da data de criação até o momento da coleta. A hipótese formulada era de que projetos mais antigos apresentariam DIT ligeiramente mais elevado — fruto da estratificação natural das hierarquias ao longo do tempo — e LCOM variável, decorrente de refatorações incompletas. A tabela abaixo apresenta as estatísticas descritivas da variável idade.

Medida	Valor
Mediana	10,26 anos
Média	10,11 anos
Desvio Padrão	3,16 anos

Tabela 7 — Estatísticas descritivas: idade dos repositórios em anos ($n = 978$).

A distribuição da idade é relativamente simétrica (mediana $\approx$ média), com baixa dispersão, indicando que os repositórios mais populares do GitHub tendem a ter uma trajetória temporal similar. Os resultados de correlação são apresentados a seguir.

Métrica de Qualidade	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,0196	0,5413	Não
DIT (mediana)	0,0982	0,0022	Sim
LCOM (mediana)	0,0906	0,0047	Sim

Tabela 8 — Correlação de Spearman: idade (anos) $\times$ métricas de qualidade.

A correlação entre maturidade e CBO não foi significativa ($p = 0,020$, $p = 0,54$). No entanto, DIT ($p = 0,098$, $p = 0,002$) e LCOM ($p = 0,091$, $p = 0,005$) apresentaram correlações fracas, porém estatisticamente significativas, no sentido positivo: projetos mais antigos tendem a ter hierarquias de herança ligeiramente mais profundas e menor coesão entre métodos. A hipótese é parcialmente confirmada. Embora o efeito seja real e significativo, sua magnitude baixa indica que a maturidade por si só não é um preditor forte de qualidade — outros fatores como tamanho e atividade de refatoração importam mais.

4.3 RQ03 — Atividade vs. Qualidade

A atividade do repositório é quantificada pelo número total de releases publicadas. Projetos sem releases formais recebem valor zero. A hipótese era de que uma cadência alta de releases reflete manutenção ativa, o que favoreceria melhores valores de CBO e LCOM ao longo do tempo. A tabela a seguir apresenta as estatísticas descritivas.

Medida	Valor
--------	-------

Mediana	11,00
Média	40,79
Desvio Padrão	89,28

Tabela 9 — Estatísticas descritivas: número de releases ($n = 978$).

A extrema assimetria da distribuição (desvio padrão mais de duas vezes superior à média, que por sua vez é quase quatro vezes a mediana) reflete que poucos projetos concentram a maioria das releases — típico de grandes frameworks com ciclos de versionamento muito ativos. Os resultados da correlação são apresentados abaixo.

Métrica de Qualidade	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,2871	< 0,001	Sim
DIT (mediana)	-0,0530	0,0993	Não
LCOM (mediana)	0,1154	0,0003	Sim

Tabela 10 — Correlação de Spearman: número de releases $\times$ métricas de qualidade.

Os resultados vão na direção oposta à hipótese: projetos com mais releases apresentam mais acoplamento (CBO, $\rho = 0,287$) e menos coesão (LCOM, $\rho = 0,115$). Uma interpretação plausível é que projetos com muitas releases são geralmente maiores e mais complexos — o crescimento funcional ao longo das versões introduz novas dependências entre classes sem que refatorações estruturais as contenham. A hipótese inicial é, portanto, refutada.

4.4 RQ04 — Tamanho vs. Qualidade

O tamanho é avaliado por duas dimensões complementares: a mediana de linhas de código por classe (LOC) e o total de linhas de comentário do repositório. A hipótese era de que projetos maiores apresentariam CBO e DIT mais elevados, por conta de maior complexidade estrutural. As tabelas a seguir apresentam as estatísticas descritivas de ambas as dimensões.

LOC — Linhas de Código (mediana por classe)

Medida	Valor
Mediana	17,00 linhas
Média	19,29 linhas
Desvio Padrão	13,23 linhas

Tabela 11 — Estatísticas descritivas: LOC mediana por classe ($n = 978$).

Linhas de Comentário (total por repositório)

Medida	Valor
--------	-------

Mediana	4.757,50
Média	30.039,06
Desvio Padrão	75.268,15

Tabela 12 — Estatísticas descritivas: linhas de comentário totais ($n = 978$).

O alto desvio padrão de linhas de comentário ($dp = 75.268$) evidencia grande heterogeneidade: alguns projetos comentam extensivamente (possivelmente documentação de API pública), enquanto a maioria mantém poucas linhas. Os resultados das correlações são apresentados nas tabelas a seguir.

Correlações: LOC × Qualidade

Métrica de Qualidade	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,4292	< 0,001	Sim
DIT (mediana)	0,1322	< 0,001	Sim
LCOM (mediana)	0,5491	< 0,001	Sim

Tabela 13 — Correlação de Spearman: LOC mediana × métricas de qualidade.

Correlações: Linhas de Comentário × Qualidade

Métrica de Qualidade	ρ (Spearman)	p-value	Significativo?
CBO (mediana)	0,3051	< 0,001	Sim
DIT (mediana)	-0,0479	0,1358	Não
LCOM (mediana)	0,1277	< 0,001	Sim

Tabela 14 — Correlação de Spearman: linhas de comentário × métricas de qualidade.

Tamanho apresentou as correlações mais fortes de todo o estudo. A correlação LOC vs. LCOM ($\rho = 0,549$) é moderada-forte: classes maiores tendem a ser substancialmente menos coesas, acumulando responsabilidades sem divisão clara de papéis. A correlação LOC vs. CBO ($\rho = 0,429$) é moderada, indicando que classes maiores também dependem de mais outras classes. As linhas de comentário reforçam o padrão para CBO ($\rho = 0,305$), mas com intensidade menor. A hipótese é confirmada.

5. Discussão e Visualização dos Resultados

Esta seção reúne os gráficos de dispersão gerados pelo script `analyze.py` para cada questão de pesquisa. Cada gráfico exibe o coeficiente de Spearman (ρ) e o p-value correspondente. Os gráficos devem ser inseridos nas posições indicadas abaixo, que correspondem à ordem de análise das RQs.

5.1 RQ01 — Popularidade vs. Qualidade

Os três gráficos a seguir (Figuras 2, 3 e 4) evidenciam a ausência de qualquer padrão entre o número de estrelas e as métricas de qualidade. Os pontos estão dispersos sem tendência visual clara, corroborando os valores de ρ próximos de zero e os p-values superiores a 0,05. A distribuição altamente assimétrica das estrelas — com a maioria dos repositórios concentrada entre 0 e 20.000 e poucos outliers acima de 100.000 — também contribui para dificultar qualquer correlação.

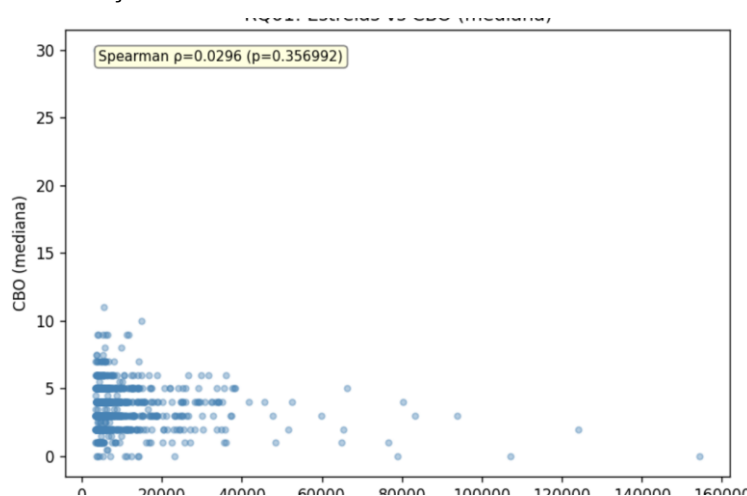


Figura 2 — Dispersão: estrelas $\times$ CBO (mediana). Spearman $\rho = 0,030$ ($p = 0,357$).

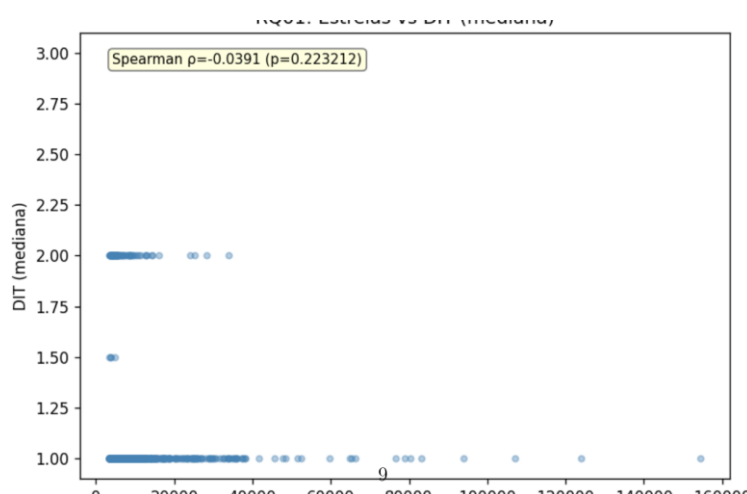


Figura 3 — Dispersão: estrelas $\times$ DIT (mediana). Spearman $\rho = -0,039$ ($p = 0,223$).

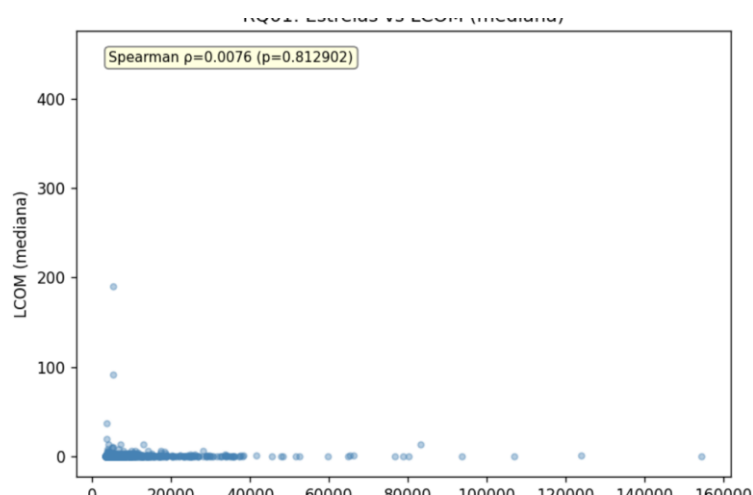


Figura 4 — Dispersão: estrelas × LCOM (mediana). Spearman $\rho = 0,008$ ($p = 0,813$).

5.2 RQ02 — Maturidade vs. Qualidade

As Figuras 5, 6 e 7 mostram que os dados de idade se distribuem de forma mais homogênea que as estrelas, com a maioria dos repositórios entre 5 e 15 anos. Para CBO, a nuvem de pontos não apresenta inclinação visível, consistente com a correlação não significativa. Para DIT e LCOM, uma leve tendência positiva pode ser percebida nos repositórios com mais de 10 anos, embora a magnitude do efeito seja pequena.

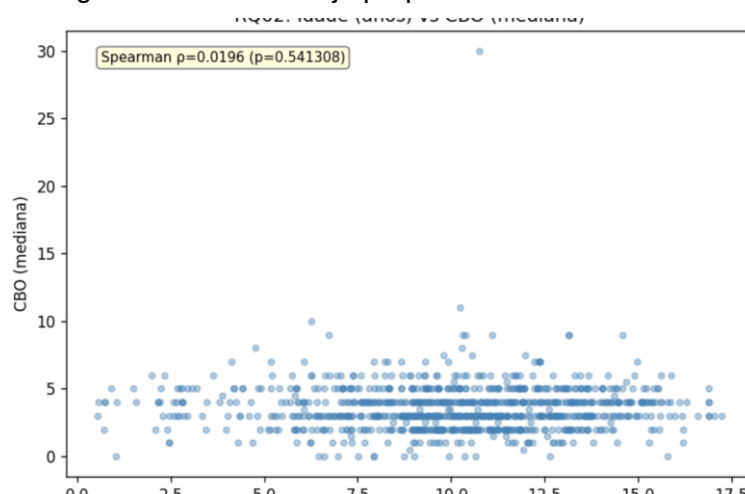


Figura 5 — Dispersão: idade (anos) × CBO (mediana). Spearman $\rho = 0,020$ ($p = 0,541$).



Figura 6 — Dispersão: idade (anos) × DIT (mediana). Spearman $\rho = 0,098$ ($p < 0,05$).

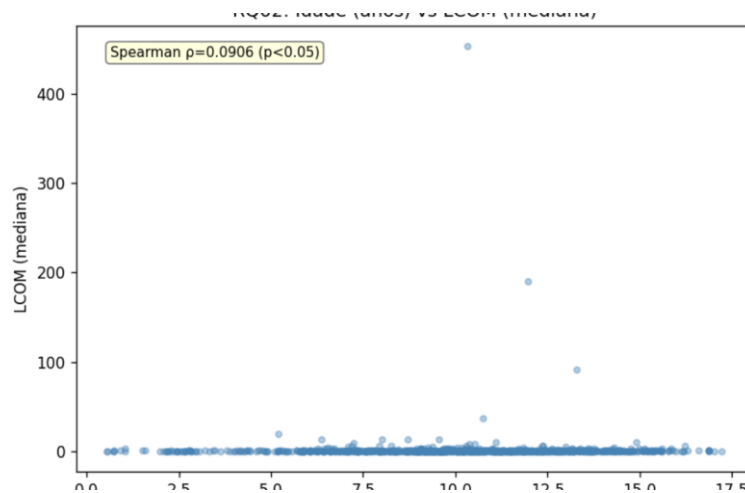


Figura 7 — Dispersão: idade (anos) × LCOM (mediana). Spearman $\rho = 0,091$ ($p < 0,05$).

5.3 RQ03 — Atividade vs. Qualidade

As Figuras 8, 9 e 10 revelam que a grande maioria dos repositórios possui entre 0 e 100 releases, com poucos outliers acima de 500. Para CBO, uma inclinação positiva moderada é visível na faixa de 0 a 300 releases, refletindo o $\rho = 0,287$ obtido. Para DIT, os pontos estão horizontalmente distribuídos sem padrão claro, em linha com a correlação não significativa. Para LCOM, a tendência positiva existe, mas é ofuscada pela concentração de valores próximos de zero.

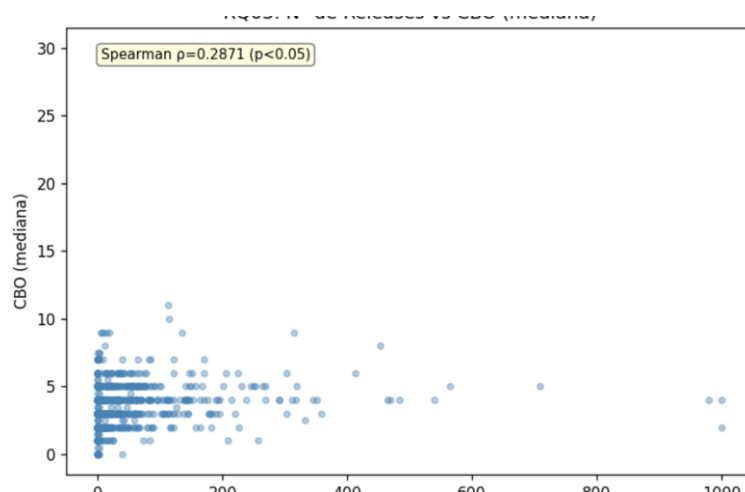


Figura 8 — Dispersão: n° releases × CBO (mediana). Spearman $\rho = 0,287$ ($p < 0,001$).

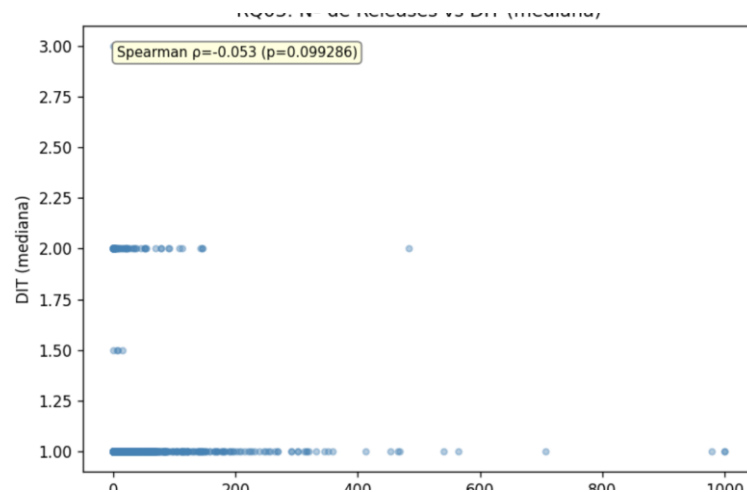


Figura 9 — Dispersão: nº releases × DIT (mediana). Spearman $\rho = -0,053$ ($p = 0,099$).

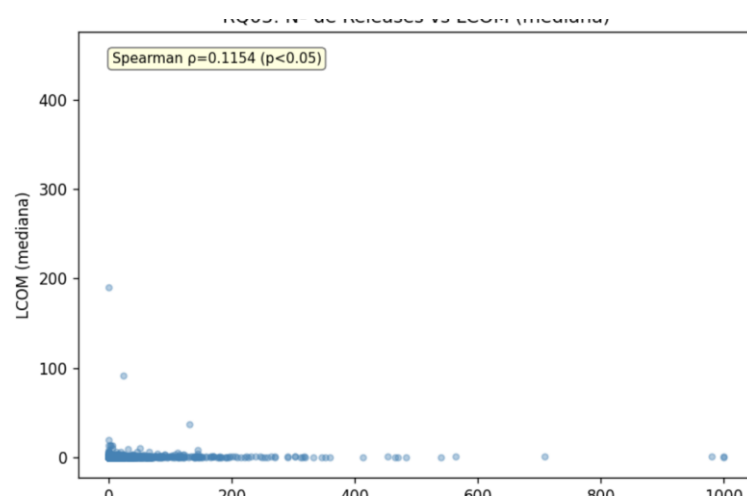


Figura 10 — Dispersão: nº releases × LCOM (mediana). Spearman $\rho = 0,115$ ($p < 0,001$).

5.4 RQ04 — Tamanho vs. Qualidade

As Figuras 11 a 16 apresentam as correlações entre tamanho (LOC e linhas de comentário) e as métricas de qualidade. O gráfico LOC × LCOM (Figura 13) é o mais expressivo do estudo: a tendência positiva é claramente visível, com $p = 0,549$ — a correlação mais forte observada. O gráfico LOC × CBO (Figura 11) também exibe inclinação positiva notável. As correlações com linhas de comentário seguem o mesmo padrão, porém com intensidade moderada, possivelmente por conta da alta variabilidade dos dados ($dp = 75.268$).

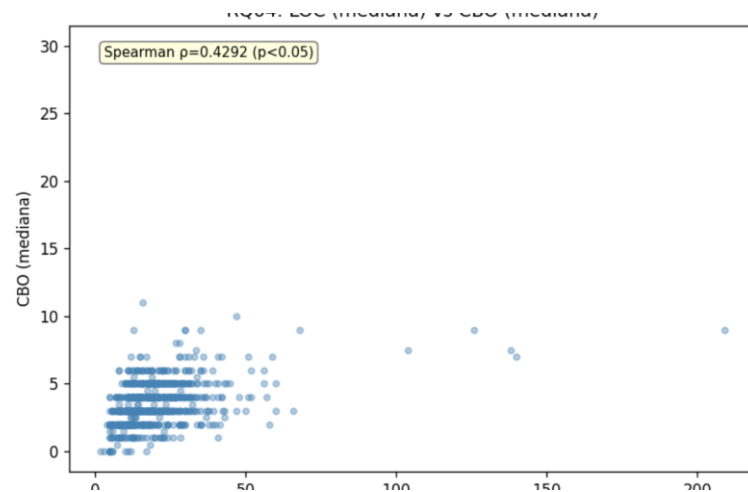


Figura 11 — Dispersão: LOC mediana $\times$ CBO (mediana). Spearman $\rho = 0,429$ ($p < 0,001$).

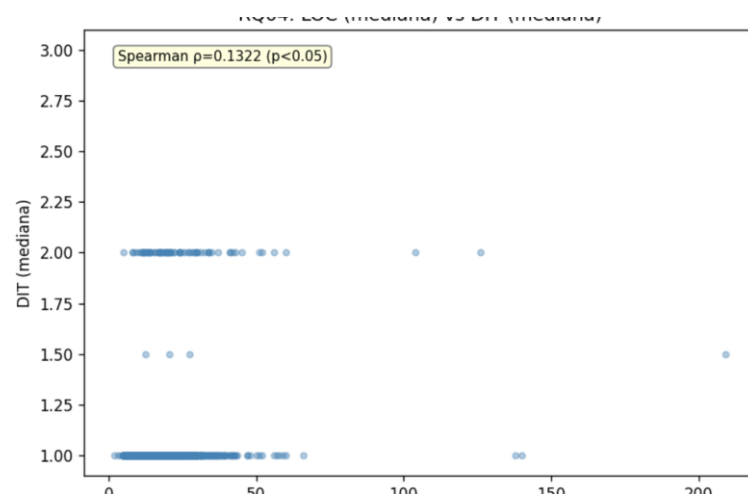


Figura 12 — Dispersão: LOC mediana $\times$ DIT (mediana). Spearman $\rho = 0,132$ ($p < 0,001$).

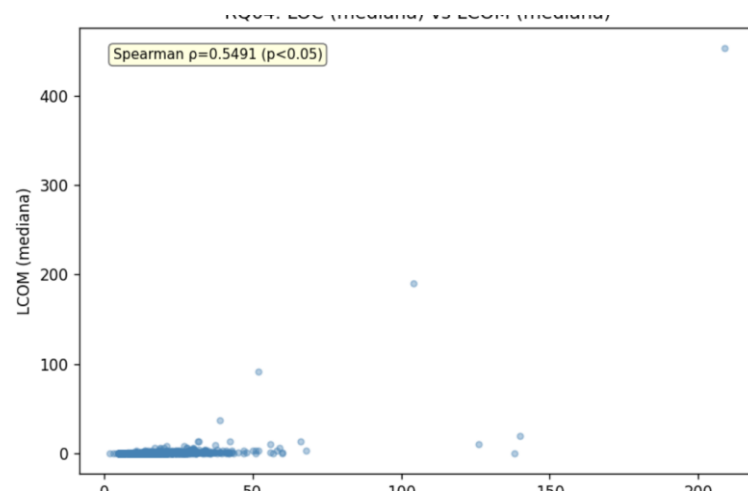


Figura 13 — Dispersão: LOC mediana $\times$ LCOM (mediana). Spearman $\rho = 0,549$ ($p < 0,001$).

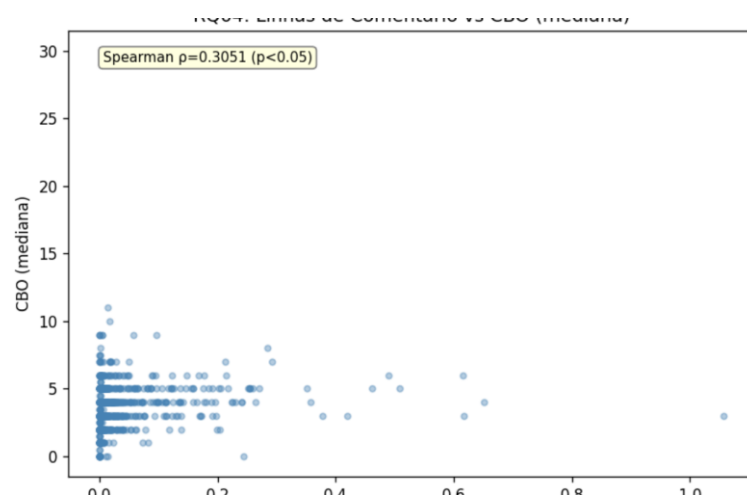


Figura 14 — Dispersão: linhas de comentário × CBO (mediana). Spearman $\rho = 0,305$ ($p < 0,001$).

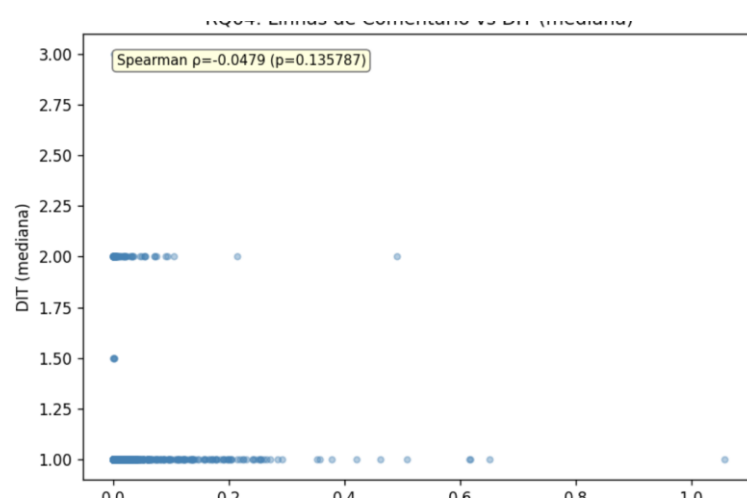


Figura 15 — Dispersão: linhas de comentário × DIT (mediana). Spearman $\rho = -0,048$ ($p = 0,136$).

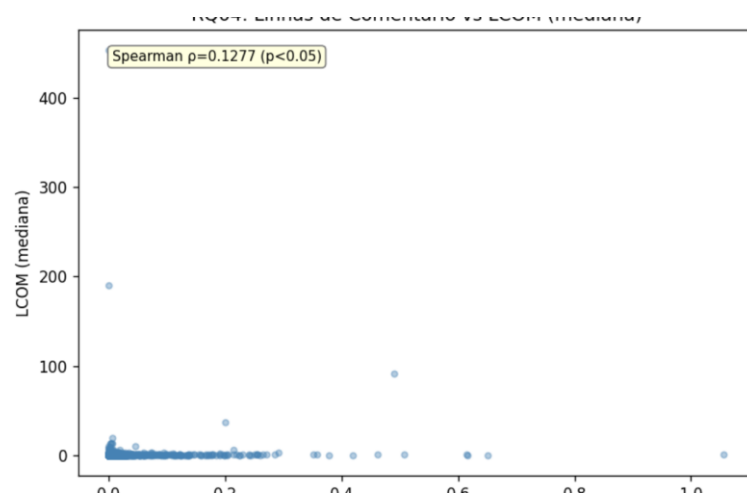


Figura 16 — Dispersão: linhas de comentário × LCOM (mediana). Spearman $\rho = 0,128$ ($p < 0,001$).

6. Conclusão

Este estudo analisou empiricamente a qualidade interna de 978 repositórios Java populares no GitHub, correlacionando métricas de processo com métricas de qualidade calculadas pela ferramenta CK. A tabela a seguir sintetiza o resultado de cada questão de pesquisa em relação à hipótese formulada.

RQ	Hipótese	Resultado	Confirmada?
RQ01	Maior popularidade → menor CBO e LCOM	Correlações negligenciáveis ($\rho \approx 0$), não significativas	Não
RQ02	Maior maturidade → DIT e LCOM ligeiramente maiores	DIT ($\rho=0,098$) e LCOM ($\rho=0,091$) significativos; CBO não	Parcialmente
RQ03	Mais releases → melhor CBO e LCOM	CBO e LCOM aumentam com releases — sentido oposto à hipótese	Não
RQ04	Maior tamanho → CBO e LCOM mais altos	LOC vs LCOM ($\rho=0,549$) e LOC vs CBO ($\rho=0,429$) — confirmado	Sim

Tabela 15 — Síntese dos resultados por questão de pesquisa.

Os quatro achados principais do estudo são:

- **Popularidade não implica qualidade interna.** Estrelas medem visibilidade e adoção, não arquitetura. A ausência de correlação sugere que projetos educacionais e frameworks amplamente adotados coexistem na amostra com perfis de qualidade estrutural muito distintos.
- **Tamanho é o preditor mais forte de degradação de qualidade.** A correlação LOC vs. LCOM ($\rho = 0,549$) é a mais alta do estudo, indicando que classes maiores acumulam responsabilidades de forma desproporcional.
- **Projetos mais ativos tendem a ser maiores e, por isso, exibem mais acoplamento.** A correlação atividade vs. CBO ($\rho = 0,287$) reflete o crescimento acumulado de dependências ao longo das releases, sem refatoração estrutural correspondente.
- **Maturidade introduz leve acúmulo de dívida técnica.** As correlações significativas mas fracas de DIT ($\rho = 0,098$) e LCOM ($\rho = 0,091$) com a idade sugerem que projetos mais antigos acumulam gradualmente hierarquias mais profundas e menor coesão.

Como limitações do estudo, destaca-se a exclusão de repositórios que utilizam sintaxe Java 14+ incompatível com o CK 0.7.0 — projetos mais recentes e possivelmente mais bem estruturados podem estar sub-representados na amostra. Adicionalmente, a análise representa uma fotografia temporal dos repositórios no momento da coleta, e os resultados podem variar conforme a janela de tempo adotada.